

Coding Hands-on — Medium: Trie-Indexed String

Problem Statement

You are given a string S (lowercase letters) and a dictionary of W words. Partition S into the maximum number of non-overlapping substrings, where each substring must be a word in the dictionary.

Words may be reused.

If S cannot be fully partitioned using dictionary words, consider the maximum prefix of S that can be fully partitioned.

Total score = $\text{max_parts} \times 1000 + \text{max_prefix_len}$, where:

- max_parts is the maximum number of substrings.
- max_prefix_len is the length of the longest prefix that can be fully partitioned.

Find the **maximum total score achievable**.

Input Format

- The first line contains a string S , denoting the input string.
- The second line contains an integer W , denoting the number of words.
- Each of the next W lines contains a string $\text{words}[i]$.

Constraints

- $1 \leq |S| \leq 500$
- $1 \leq W \leq 200$
- $1 \leq |\text{words}[i]| \leq 20$

Sample Test Cases

Case 1

Input

aaba
3

a
aa
b

Output

4004

Explanation

The string **aaba** (length 4) can be fully partitioned using dictionary words.
Possible partitions:

- $a + a + b + a \rightarrow 4$ parts
- $aa + b + a \rightarrow 3$ parts

Maximum parts = 4

Longest valid prefix length = 4

$$\text{Score} = 4 \times 1000 + 4 = 4004$$

Case 2

Input

hello
3
he
l
o

Output

4005

Explanation

The string **hello** can be fully partitioned as:

$he + l + l + o$

Maximum parts = 4

Longest valid prefix length = 5

$$\text{Score} = 4 \times 1000 + 5 = 4005$$

Case 3

Input

xyzabc
3
x
y

za

Output

3004

Explanation

Dictionary words can form the prefix **xyza**:

x + y + za

No longer prefix can be fully partitioned.

Maximum parts = 3

Longest valid prefix length = 4

Score = 3 × 1000 + 4 = 3004

Problem Statement

Given a string **S** consisting only of opening ' (' and closing ') ' brackets, and two integers **a** and **b**:

- Inserting one opening bracket ' (' costs **a**
- Inserting one closing bracket ') ' costs **b**

Your task is to make the bracket sequence valid by inserting the minimum number of brackets, and return the minimum total cost.

A valid bracket sequence is one where every opening bracket has a matching closing bracket in the correct order.

Function Signature

```
int solve(string s, int a, int b)
```

Input Format

- First line: string **s**
- Second line: integer **a**
- Third line: integer **b**

Sample Test Cases

Case 1

Input:

) (
2
3

Output:

5

Explanation:

The string is ") (".

- Unmatched ')' at the start $\rightarrow$ insert '(' $\rightarrow$ cost = 2
- Unmatched '(' at the end $\rightarrow$ insert ')' $\rightarrow$ cost = 3
Total = 5

Case 2

Input:

(((
10
5

Output:

15

Explanation:

There are 3 unmatched '(' brackets.

Insert 3 ')' brackets.

Total cost = $3 \times 5 = 15$

Case 3

Input:

()()
100
100

Output:

0

Explanation:

The string is already a valid bracket sequence.

2Subseqs

Problem Statement

You are given two integers **N** and **M**, along with an array **A** of **N** integers and an array **B** of **M** integers.

Count the number of ways to select a subsequence **S** from **A** and a subsequence **D** from **B** such that:

- **S** and **D** have the same length.
- When you concatenate **S** followed by **D** (in that order), the resulting sequence forms a palindrome.

Print the number of such palindrome pairs modulo $10^9 + 7$.

Input Format

- The first line contains an integer **N**, denoting the length of array **A**.
- The second line contains an integer **M**, denoting the length of array **B**.
- Each of the next **N** lines contains one integer **A[i]**.
- Each of the next **M** lines contains one integer **B[i]**.

Constraints

- $(1 \leq N \leq 3000)$
- $(1 \leq M \leq 3000)$
- $(1 \leq A[i] \leq 10^9)$
- $(1 \leq B[i] \leq 10^9)$

Sample Test Cases

Case 1

Input:

```
2
2
1
1
```

1

1

Output:

4

Explanation:

Valid palindrome pairs are:

- Empty subsequence from both arrays
- [1] from A and [1] from B (first occurrence)
- [1] from A and [1] from B (second occurrence)
- [1, 1] from A and [1, 1] from B

Case 2

Input:

2

2

1

1

2

2

Output:

1

Explanation:

Only the empty subsequence forms a valid palindrome pair.

Case 3

Input:

2

2

1

1

1

2

Output:

3

Explanation:

Valid palindrome pairs:

- Empty subsequence
- [1] from A with first matching [1] from B
- [1] from A with second possible matching choice